

AI/BigData 인텐시브 과정

강사장철원

Section 0

코스 소개

DAY1	DAY2	DAY3	DAY4	DAY5
데이터 분석을 위한 기초 수학			데이터 집계 및 시각화	
DAY6	DAY7	DAY8	DAY9	DAY10
	미니 프로젝트	확률분포, AB테스트		
DAY11	DAY12	DAY13	DAY14	DAY15
머신러닝				
DAY16	DAY17	DAY18	DAY19	DAY20
딥러닝		파이널 프로젝트		

□ 시계열 분석의 개념

□ ARIMA

□ GARCH

시계열 분석에 필요한 라이브러리

```
(py3_11_9) C:\Users\stoic> pip install statsmodels
```

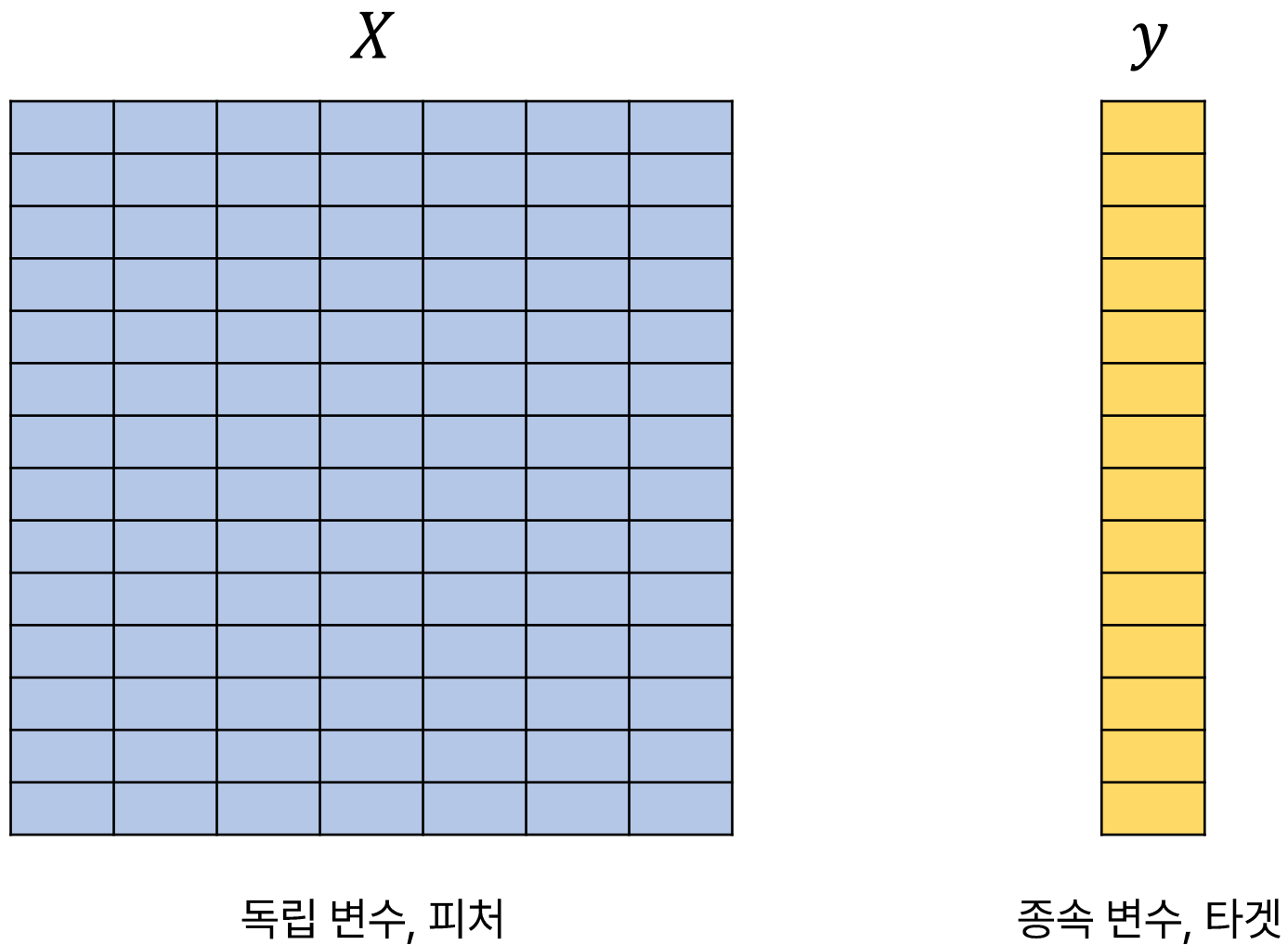
```
(py3_11_9) C:\Users\stoic> pip install arch
```

AI/BigData 인텐시브 과정

Section 1. 회귀 분석

Section 1-1. 회귀분석 개념

회귀분석의 개념



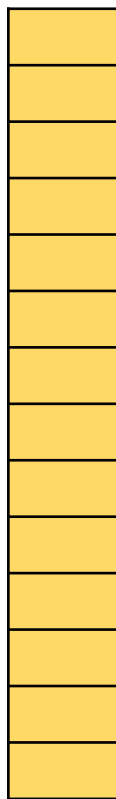
단순 회귀분석의 개념

X



독립 변수, 피처

y



종속 변수, 타겟

회귀분석의 가정

(1) 변수 x 와 y 사이에는 다음과 같은 관련성이 존재한다.

$$E(y) = \beta_1 x + \beta_0$$

$$E(y) = w_1 x + b$$

(2) 독립 변수 x 값이 주어졌을 때, 종속 변수 y 는 정규 분포를 따른다.

(3) 이 때, y 의 평균은 $E(y) = \beta_1 x + \beta_0$ 로 독립 변수 x 값에 따라 변하나 분산은 x 값에 관계 없이 일정하다.

(4) 독립 변수 x 는 오차 없이 측정할 수 있는 수학 변수(mathematical variable)이며, 종속 변수 y 는 측정 오차를 수반하는 확률 변수(random variable)이다.

(5) 종속 변수 y 의 오차들은 서로 독립이다.

회귀 분석 가정의 수식화

$$y_i = \beta_1 x + \beta_0 + \epsilon_i$$

$$y_i \sim N(\beta_1 x_i + \beta_0, \sigma^2)$$

$$\epsilon_i \sim N(0, \sigma^2)$$

$$\text{Cov}(\epsilon_i, \epsilon_j) = 0$$

$$\epsilon_i = y_i - \hat{y} = y_i - (\beta_1 x + \beta_0)$$

$$\text{Var}(y_i) = \sigma^2 \quad \text{등분산 가정}$$

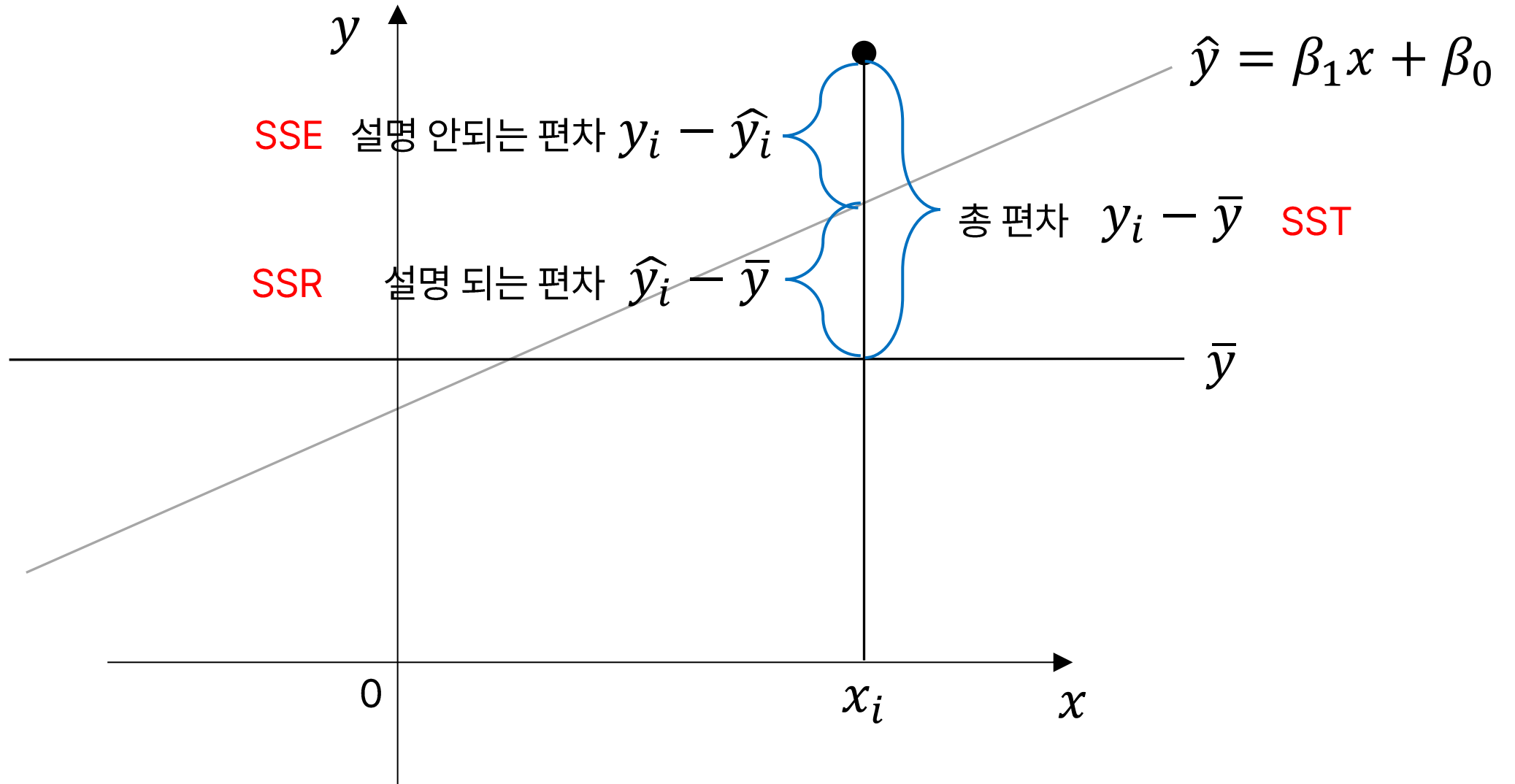
$$\text{Var}(\epsilon_i) = \sigma^2 \quad \text{등분산 가정}$$

최소 제곱 추정량(least square estimator)

$$S = \sum_{i=1}^n \epsilon_i^2 = \sum_{i=1}^n (y_i - \beta_1 x - \beta_0)^2$$

오차 제곱합을 최소로 하는 β_0, β_1 구하기

회귀 분석 가정의 수식화



회귀 분석 가정의 수식화

$$SST = SSE + SSR$$

총 변동

회귀 모형에 의해

회귀 모형에 의해

설명 안되는 변동

설명 되는 변동

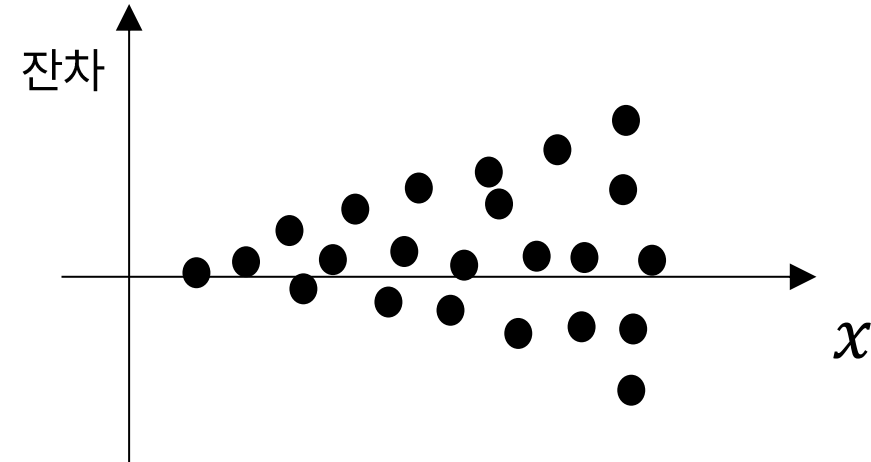
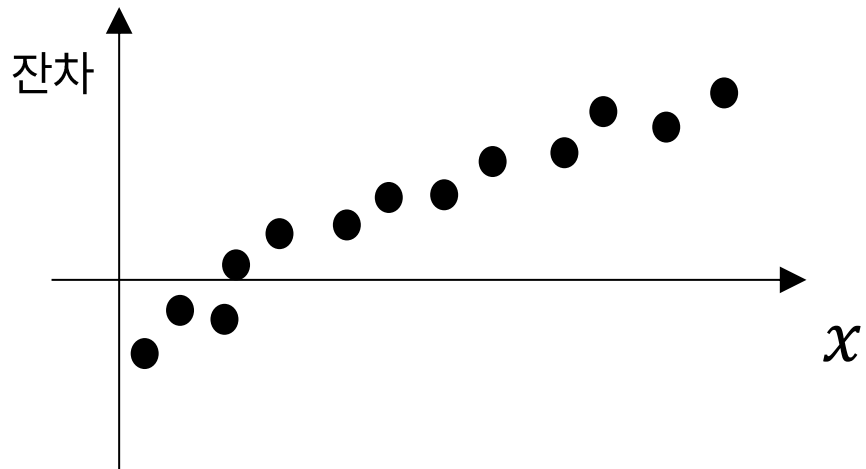
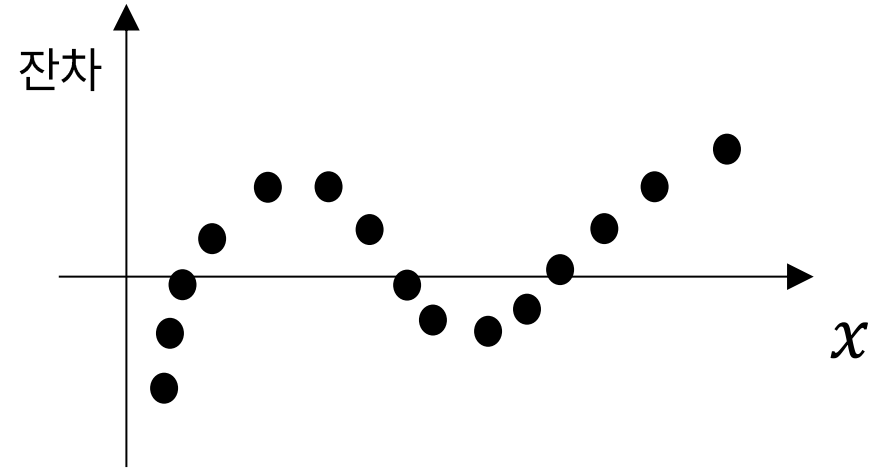
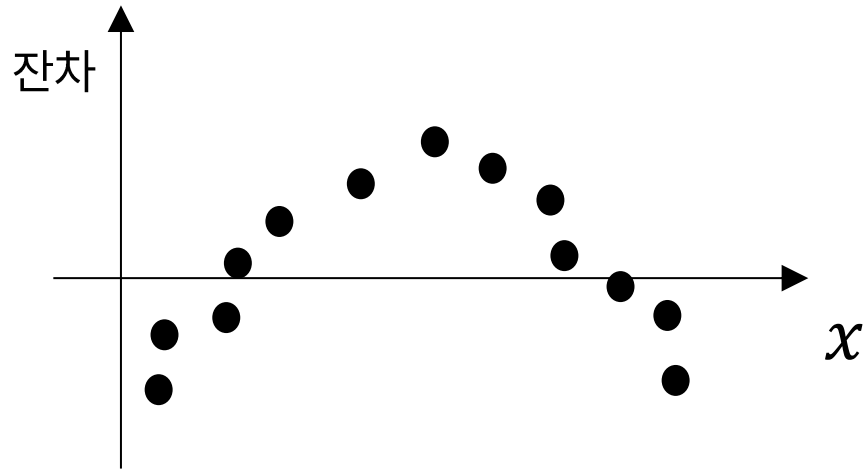
$$\sum_{i=1}^n (y_i - \bar{y})^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

결정 계수

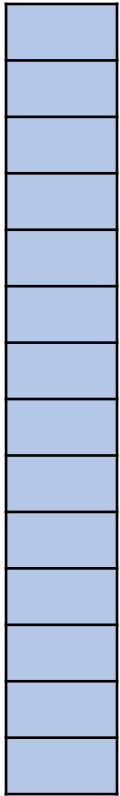
$$r^2 = \frac{SSR}{SST} \quad 0 \leq r^2 \leq 1$$

전체 변동 중 회귀 모형으로 설명가능한 변동의 비율

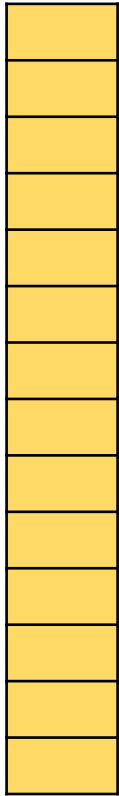
가정을 만족하지 않는 경우



단순 회귀분석의 개념

 X 

독립 변수, 피처

 y 

종속 변수, 타겟

$$y_i = \beta_1 x + \beta_0 + \epsilon_i$$

단순 회귀 모형

$$\mathbf{X} = \begin{pmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \boldsymbol{\beta} = \begin{pmatrix} \beta_1 \\ \beta_0 \end{pmatrix}$$

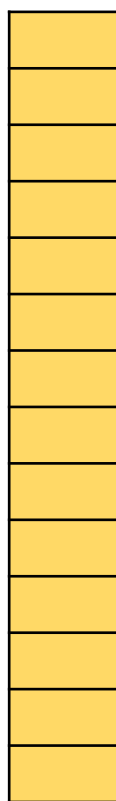
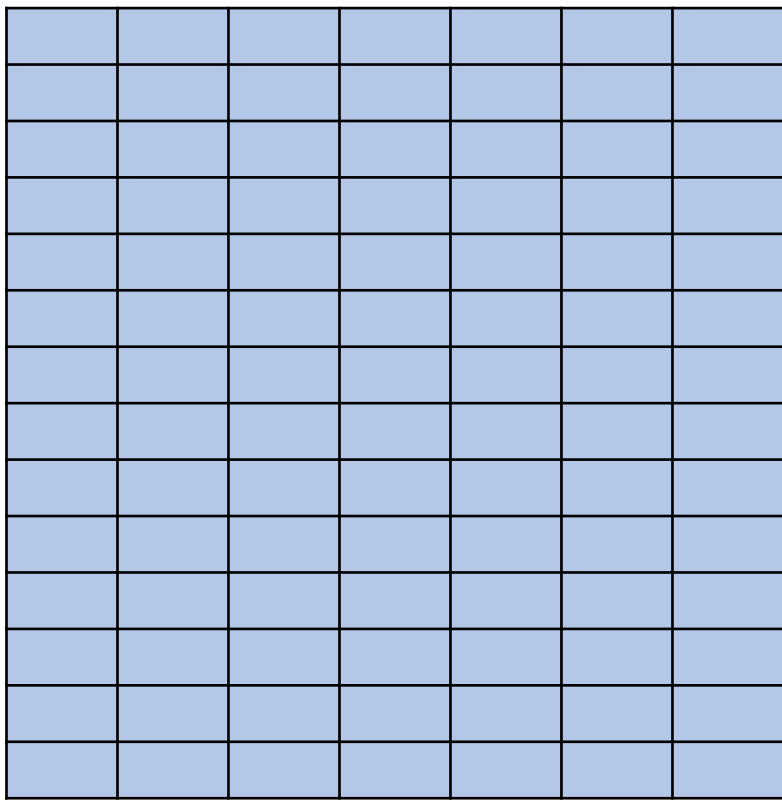
$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta}$$

단순 회귀 모형

$$\mathbf{X} = \begin{pmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} w_1 \\ b \end{pmatrix}$$

$$\mathbf{y} = \mathbf{X}\mathbf{w}$$

회귀분석의 개념

 X y 

$$y = Xw + \epsilon$$

독립 변수, 피처

종속 변수, 타겟

회귀분석의 개념

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} & 1 \\ x_{21} & x_{22} & \cdots & x_{2p} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{x}_1^T & 1 \\ \mathbf{x}_2^T & 1 \\ \vdots & \vdots \\ \mathbf{x}_n^T & 1 \end{pmatrix}$$

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} w_1 \\ w_2 \\ \vdots \\ w_p \\ b \end{pmatrix}$$

$$\mathbf{y} = \mathbf{X}\mathbf{w}$$

중 회귀 모형

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

$$\frac{\partial \mathbf{e}}{\partial \mathbf{w}} = \frac{\partial}{\partial \mathbf{w}} (\mathbf{y}^T \mathbf{y} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}) = -2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X} \mathbf{w}$$

$$\hat{\mathbf{w}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

$$\mathbf{e} = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

$$= (\mathbf{y}^T - \mathbf{w}^T \mathbf{X}^T) (\mathbf{y} - \mathbf{X}\mathbf{w})$$

$$= \mathbf{y}^T \mathbf{y} - \mathbf{y}^T \mathbf{X} \mathbf{w} - \mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}$$

$$= \mathbf{y}^T \mathbf{y} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w}$$

다중공선성(multicollinearity)

 X

독립 변수, 피쳐

어떤 독립 변수가 다른 독립 변수들의 선형 조합으로 표현될 수 있는 경우

$$\text{EX) } x_1 = x_2 + x_3$$

AI/BigData 인텐시브 과정

Section 1. 회귀 분석

Section 1-2. 회귀분석 실습

라이브러리 불러오기

In [1]: ▶

```
1 import numpy as np
2 import pandas as pd
3 import statsmodels
4 from statsmodels.regression import linear_model
5 from statsmodels.tsa.arima.model import ARIMA
6 from arch import arch_model
```

회귀분석

ARIMA

ARCH, GARCH

데이터 불러오기

```
In [2]: 1 df = pd.read_csv('./data/air_quality_uci.csv', encoding='cp949')
```

```
In [3]: 1 df.head(3)
```

Out[3]:

	Date	Time	CO	PT08_S1	NMHC	C6H6	PT08_S2	NOx	PT08_S3	NO2	PT08_S4	PT08_S5	T	RH	AH
0	2005-02-16	17:00:00	1.6	1083	-200	7.0	856	252	720	148	1010	975	8.2	41.8	0.4563
1	2005-02-16	18:00:00	2.2	1143	-200	9.5	960	313	641	153	1110	1136	7.4	44.5	0.4582
2	2005-02-16	19:00:00	3.4	1288	-200	15.9	1178	433	539	171	1300	1434	7.0	43.4	0.4366

```
In [4]: 1 df.shape
```

Out[4]: (1000, 15)

DateTime 생성

```
In [5]: 1 df['DateTime'] = df['Date'] + ' ' + df['Time']
```

```
In [6]: 1 df.head(3)
```

Out[6]:

Date	Time	CO	PT08_S1	NMHC	C6H6	PT08_S2	NOx	PT08_S3	NO2	PT08_S4	PT08_S5	T	RH	AH	DateTime
2005-02-16	17:00:00	1.6	1083	-200	7.0	856	252	720	148	1010	975	8.2	41.8	0.4563	2005-02-16 17:00:00
2005-02-16	18:00:00	2.2	1143	-200	9.5	960	313	641	153	1110	1136	7.4	44.5	0.4582	2005-02-16 18:00:00
2005-02-16	19:00:00	3.4	1288	-200	15.9	1178	433	539	171	1300	1434	7.0	43.4	0.4366	2005-02-16 19:00:00

```
In [7]: 1 df['DateTime'][3]    DateTime 피처에 값이 제대로 들어갔는지 확인
```

Out[7]: '2005-02-16 20:00:00'

```
In [8]: 1 type(df['DateTime'][3])    DateTime 피처의 값 타입을 확인한 결과 문자열인 것을 확인 -> datetime 형태로 변경 필요
```

Out[8]: str

DateTimeIndex 피쳐 생성 후 인덱스 적용

```
In [9]: 1 df['DateTimeIndex'] = pd.to_datetime(df['DateTime'])
```

```
In [10]: 1 df.head(3)
```

Out[10]:

	CO	PT08_S1	NMHC	C6H6	PT08_S2	NOx	PT08_S3	NO2	PT08_S4	PT08_S5	T	RH	AH	DateTime	DateTimeIndex
0	1.6	1083	-200	7.0	856	252	720	148	1010	975	8.2	41.8	0.4563	2005-02-16 17:00:00	2005-02-16 17:00:00
1	2.2	1143	-200	9.5	960	313	641	153	1110	1136	7.4	44.5	0.4582	2005-02-16 18:00:00	2005-02-16 18:00:00
2	3.4	1288	-200	15.9	1178	433	539	171	1300	1434	7.0	43.4	0.4366	2005-02-16 19:00:00	2005-02-16 19:00:00

```
In [11]: 1 # DateTimeIndex 구성 원소가 Timestamp라는 것 확인
          2 type(df['DateTimeIndex'][3])
```

Out[11]: pandas._libs.tslibs.timestamps.Timestamp

DateTimeIndex 피처를 인덱스로 적용

```
In [12]: 1 df = df.set_index(keys=['DateTimeIndex'])
```

```
In [13]: 1 # DateTimeIndex가 인덱스로 적용된 것 확인  
2 df.head(3)
```

Out[13]:

	Date	Time	CO	PT08_S1	NMHC	C6H6	PT08_S2	NOx	PT08_S3	NO2	PT08_S4	PT08_S5	T	RH
DateTimeIndex														
2005-02-16 17:00:00	2005-02-16	17:00:00	1.6	1083	-200	7.0	856	252	720	148	1010	975	8.2	41.8
2005-02-16 18:00:00	2005-02-16	18:00:00	2.2	1143	-200	9.5	960	313	641	153	1110	1136	7.4	44.5
2005-02-16 19:00:00	2005-02-16	19:00:00	3.4	1288	-200	15.9	1178	433	539	171	1300	1434	7.0	43.4

시계열 분석 라이브러리를 사용하기 위해서는 인덱스가
DateTime 형태여야 합니다.

결측치 확인

```
In [14]: 1 df.isnull().sum()
```

```
Out[14]: Date      0  
Time      0  
CO        0  
PT08_S1   0  
NMHC      0  
C6H6      0  
PT08_S2   0  
NOx       0  
PT08_S3   0  
NO2       0  
PT08_S4   0  
PT08_S5   0  
T         0  
RH        0  
AH        0  
DateTime  0  
dtype: int64
```

결측치가 존재하지 않음

트레이닝/테스트 데이터 분할

```
In [15]: ▶ 1 train_ratio = 0.9  
          2 n = len(df)  
          3 n_train = int(n*train_ratio)  
          4 df_train = df[:n_train]  
          5 df_test = df[n_train:]
```

```
In [16]: ▶ 1 n
```

Out[16]: 1000

```
In [17]: ▶ 1 len(df_train)
```

Out[17]: 900

```
In [18]: ▶ 1 len(df_test)
```

Out[18]: 100

피처/타겟 설정 및 피처/타겟 데이터 분할

In [19]: 1 df.columns

Out[19]: Index(['Date', 'Time', 'CO', 'PT08_S1', 'NMHC', 'C6H6', 'PT08_S2', 'NOx',
'PT08_S3', 'NO2', 'PT08_S4', 'PT08_S5', 'T', 'RH', 'AH', 'DateTime'],
dtype='object')

In [20]: 1 features = ['T', 'AH']
2 target = 'PT08_S4'
3
4 X_tn = df_train[features]
5 y_tn = df_train[target]
6 X_te = df_test[features]
7 y_te = df_test[target]

CO	PT08_S1	NMHC	C6H6	PT08_S2	NOx	PT08_S3	NO2	PT08_S4	PT08_S5	T	RH	AH
1.6	1083	-200	7.0	856	252	720	148	1010	975	8.2	41.8	0.4563
2.2	1143	-200	9.5	960	313	641	153	1110	1136	7.4	44.5	0.4582
3.4	1288	-200	15.9	1178	433	539	171	1300	1434	7.0	43.4	0.4366

타겟

피처1

피처2

회귀 분석 학습

In [21]: ▶

```
1 ## endog = 타겟  
2 ## exog = 피처
```

In [22]: ▶

```
1 ols_reg = linear_model.OLS(endog=y_tn, exog=X_tn)  
2 ols_model = ols_reg.fit()
```

회귀 분석 모형

In [24]:

```
1 ols_model.params
```

Out[24]:

```
T      92.087713
AH     -87.960934
dtype: float64
```

```
1 ols_model.summary()
```

OLS Regression Results

Dep. Variable:	PT08_S4	R-squared (uncentered):	0.791
Model:	OLS	Adj. R-squared (uncentered):	0.791
Method:	Least Squares	F-statistic:	1701.
Date:	Wed, 31 Jan 2024	Prob (F-statistic):	3.95e-306
Time:	23:07:03	Log-Likelihood:	-6942.3
No. Observations:	900	AIC:	1.389e+04
Df Residuals:	898	BIC:	1.390e+04
Df Model:	2		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
T	92.0877	1.590	57.911	0.000	88.967	95.209
AH	-87.9609	3.266	-26.930	0.000	-94.371	-81.550

statsmodels의 OLS 모형은 기본적으로 상수항을 포함하지 않음

$$y_i = w_0 + w_1 x_T + w_2 x_{AH} + \epsilon_i$$



$$y_i = w_1 x_T + w_2 x_{AH} + \epsilon_i$$



$$y_i = 92.08x_T - 87.96x_{AH} + \epsilon_i$$

train 데이터를 활용한 예측

```
1 ols_model.predict()
```

```
2.46223741e+02, 2.46874652e+02, 1.64479496e+02, 1.73829004e+02,  
1.37354559e+02, 1.37205026e+02, 4.51085166e+01, 1.80363567e+01,  
1.55226744e+02, 2.46751507e+02, 3.47335507e+02, 6.41532403e+02,  
9.91043499e+02, 1.22609110e+03, 1.09788958e+03, 1.13752182e+03,  
1.20372485e+03, 1.02828320e+03, 9.63575515e+02, 9.16300206e+02,  
8.51099935e+02, 7.95275561e+02, 8.13077377e+02, 8.19189924e+02,  
7.37815114e+02, 6.71664865e+02, 6.62966267e+02, 7.10267965e+02,  
7.26257786e+02, 6.84954285e+02, 6.66264064e+02, 6.85719545e+02,  
7.59776744e+02, 8.78452831e+02, 9.24778163e+02, 7.80691638e+02,  
6.64829562e+02, 6.75208214e+02, 7.14418244e+02, 7.35404246e+02,  
7.18763514e+02, 6.81972410e+02, 5.98046733e+02, 6.27590595e+02,
```

test 데이터를 활용한 예측

In [26]: ▶

```
1 ols_model.predict(X_te)
```

Out[26]: DateTimeIndex

2005-03-26 05:00:00	1387.973531
2005-03-26 06:00:00	1370.558743
2005-03-26 07:00:00	1371.579090
2005-03-26 08:00:00	1408.150292
2005-03-26 09:00:00	1489.956110
...	
2005-03-30 04:00:00	973.827329
2005-03-30 05:00:00	854.042934
2005-03-30 06:00:00	928.926965
2005-03-30 07:00:00	902.312202
2005-03-30 08:00:00	1030.707234

Length: 100, dtype: float64

AI/BigData 인텐시브 과정

Section 2. ARIMA

Section 2-1. 시계열 분석의 필요성

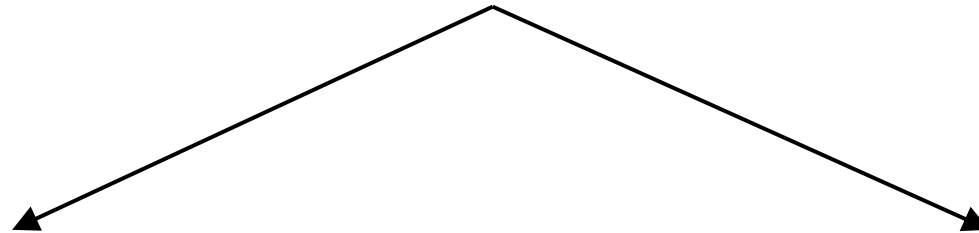
회귀분석 가정이 깨지면?

시계열 분석!

time series analysis

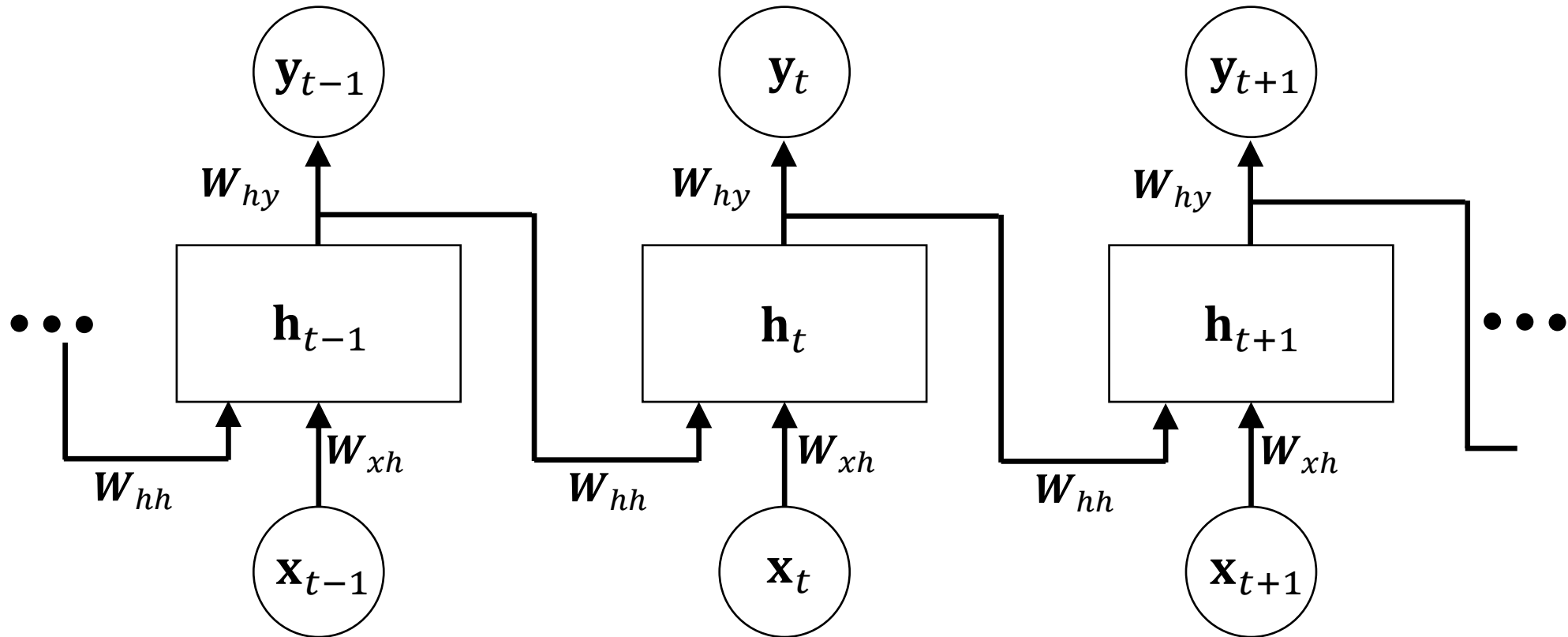
통계

딥러닝



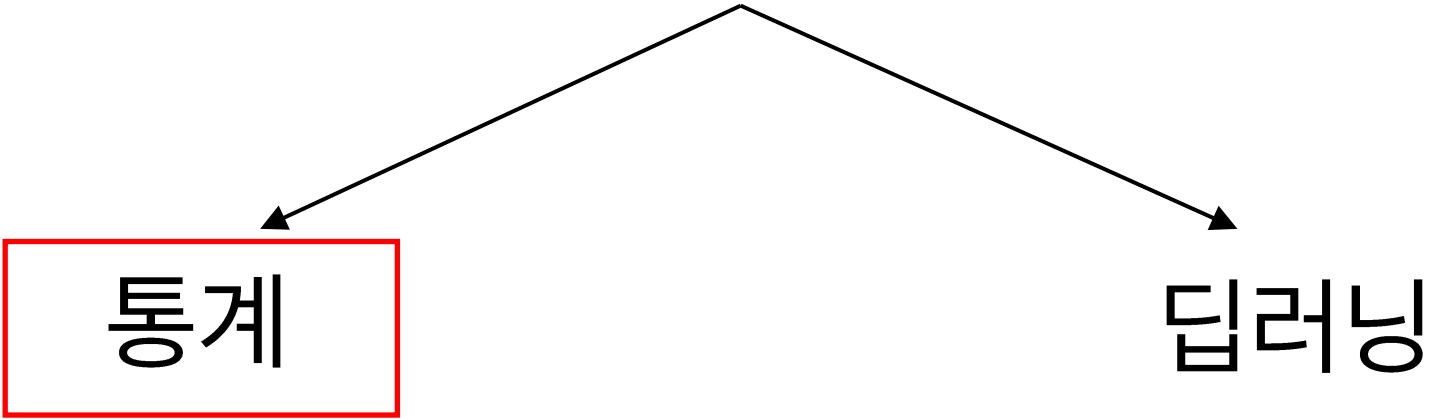
LSTM

- 딥러닝을 이용해 시계열 분석을 할 때는 LSTM과 같은 RNN 계열을 많이 활용함



회귀분석 가정이 깨지면?

시계열 분석!
time series analysis

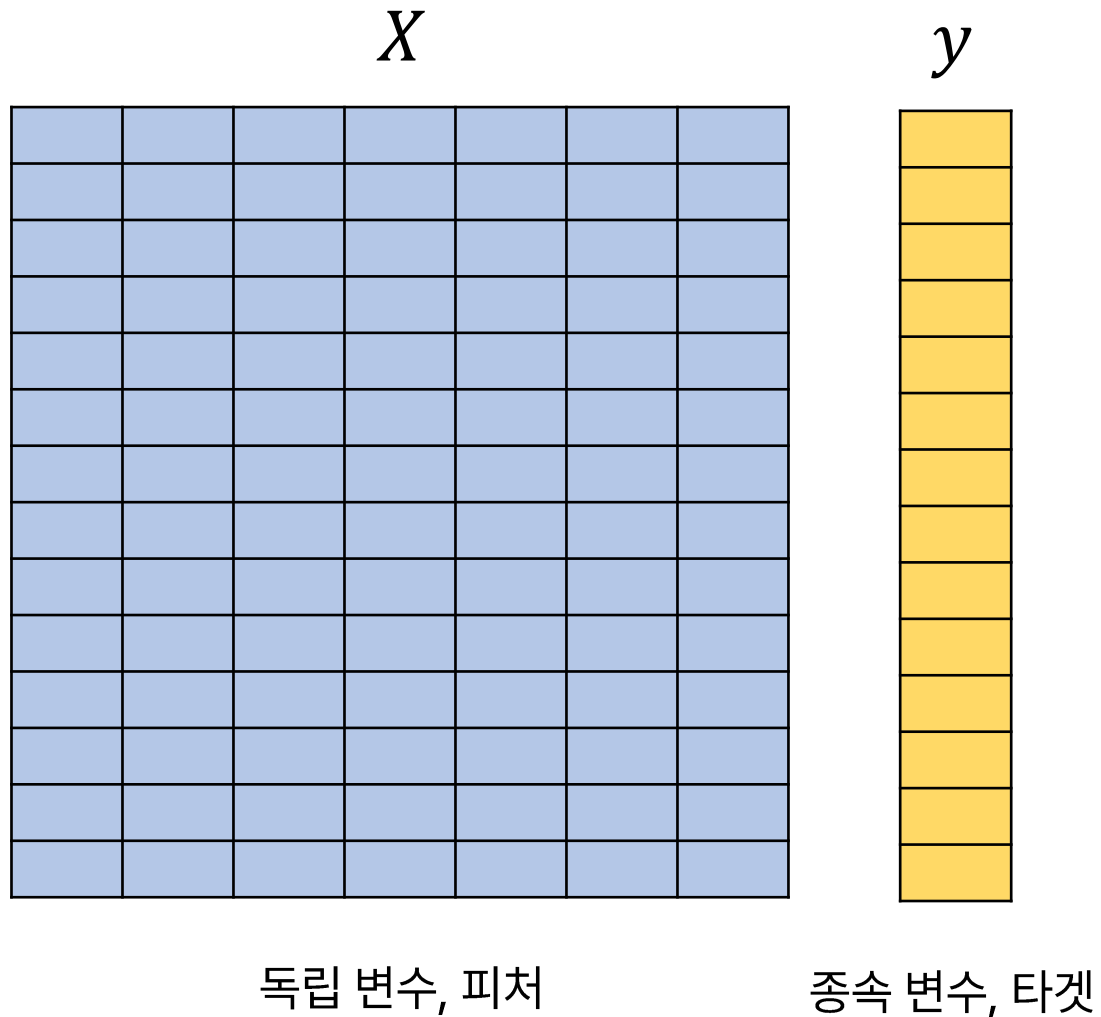


```
graph TD; A[시계열 분석!  
time series analysis] --> B[통계]; A --> C[딥러닝];
```

통계

딥러닝

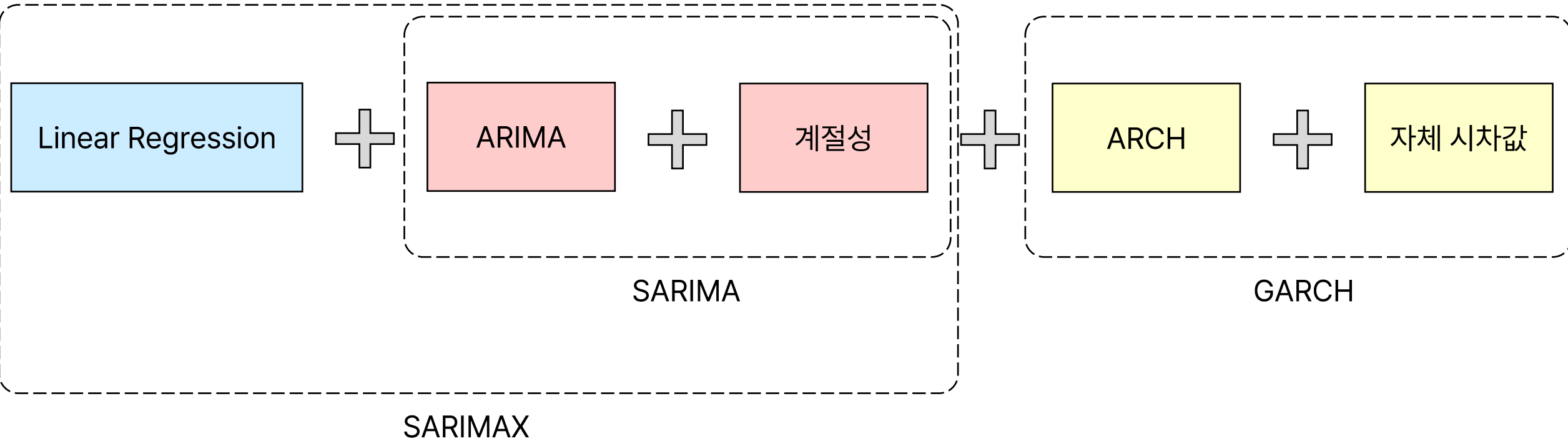
회귀분석의 개념에서 시계열 분석으로 확장



$$y = Xw + \epsilon$$

이 부분을 시계열 분석으로 예측

시계열 분석

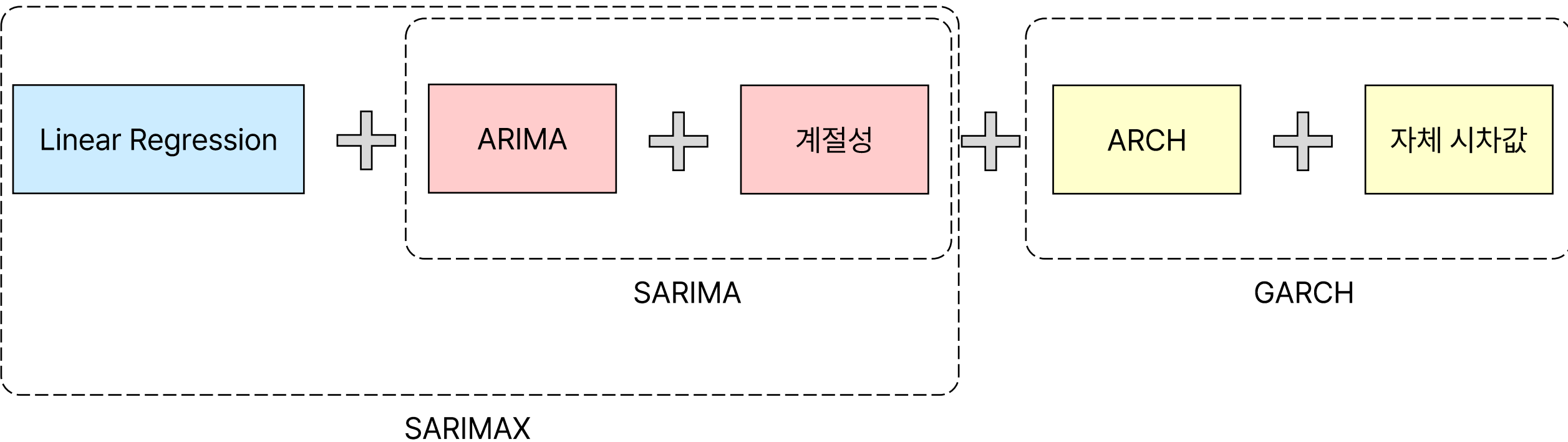


AI/BigData 인텐시브 과정

Section 2. ARIMA

Section 2-2. ARIMA의 개념

시계열 분석



기본적인 표기법

backward shift operator

$$\mathbf{B}Y_t = Y_{t-1}$$

$$\mathbf{B}^m Y_t = Y_{t-m}$$

forward shift operator

$$\mathbf{F}Y_t = Y_{t+1}$$

$$\mathbf{F}^m Y_t = Y_{t+m}$$

$$\mathbf{F} = \mathbf{B}^{-1}$$

backward difference operator

$$\begin{aligned}\nabla Y_t &= Y_t - Y_{t-1} \\ &= (1 - \mathbf{B})Y_t\end{aligned}$$

정상성(stationarity)이란?

- 시계열의 확률적인 성질들이 시간의 흐름에 따라 불변하는 것
- 평균, 분산 등이 시간의 흐름에 따라 체계적인 변화가 없고 주기적인 변화가 없는것
- 대부분의 시계열 이론들은 정상성을 가정하고 전개됨
- 따라서 정상성을 만족하지 않는 데이터는 정상성을 만족하도록 변환 필요

강한 정상성(strong stationarity)

$$f(Y_1, Y_2, \dots, Y_n) = f(Y_{1+k}, Y_{2+k}, \dots, Y_{n+k},)$$

- 시간 축을 k만큼 이동해도 결합밀도함수가 동일함

약한 정상성(weak stationarity)

$$\mu = E(Y_t) \quad \text{평균은 시간 } t \text{에 상관 없이 동일}$$

$$\sigma^2 = Var(Y_t) \quad \text{분산은 시간 } t \text{에 상관 없이 동일}$$

$$\gamma_k = Cov(Y_t, Y_{t+k}) = E[(Y_t - \mu)(Y_{t+k} - \mu)]$$

autocovariance

시차 k 에만 의존하고 시점 t 와는 무관

$$\rho_k = \frac{\gamma_k}{\gamma_0}$$

autocorrelation

백색잡음과정(white noise process)

$$Y_t = a_t$$

white noise process

$$a_t \sim WN(0, \sigma_a^2)$$

white noise process의 분포

$$E(Y_t) = E(a_t) = 0$$

white noise process의 평균

$$\sigma^2 = \gamma_0 = Var(Y_t) = Var(\epsilon_t) = \sigma_a^2$$

white noise process의 분산

$$\gamma_k = Cov(Y_t, Y_{t+k}) = 0$$

white noise process의 autocovariance

자기 회귀 과정(Auto Regressive process, AR)

- 현 시점의 값을 과거 시점의 값들,
즉 과거의 자기 자신의 관측값과 현 시점의 오차의 함수 형태로 나타내고자 함

$$AR(1) \quad Y_t = \phi_1 Y_{t-1} + a_t$$

현 시점(t)의 값 한 시점 이전(t-1)의 값 현 시점(t)의 오차

$$AR(2) \quad Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + a_t$$

현 시점(t)의 값 한 시점 이전(t-1)의 값 두 시점 이전(t-2)의 값 현 시점(t)의 오차

자기 회귀 과정(Auto Regressive process, AR)

$AR(p)$

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p} + a_t$$

$$\longleftrightarrow (1 - \phi_1 \mathbf{B} - \phi_2 \mathbf{B}^2 - \cdots - \phi_p \mathbf{B}^p) Y_t = a_t$$

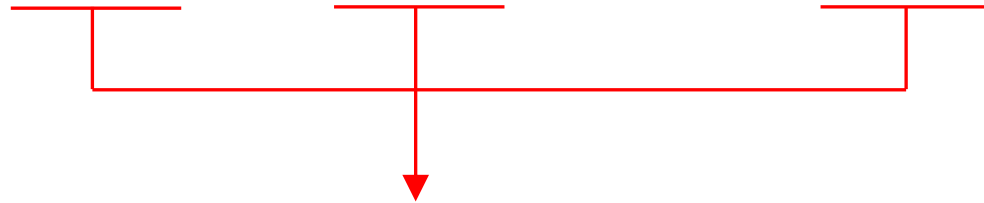
$$\longleftrightarrow \phi(\mathbf{B}) Y_t = a_t$$

$$Y_t = \frac{1}{\phi(\mathbf{B})} a_t = \phi^{-1}(\mathbf{B}) a_t \equiv \psi(\mathbf{B}) a_t \quad , \text{let } \phi^{-1} = \psi(\mathbf{B})$$

AR 모형과 회귀분석의 차이

회귀분석

$$Y = w_1x_1 + w_2x_2 + \cdots + w_px_p + a$$



AR 모형 느낌으로 바꾸면?

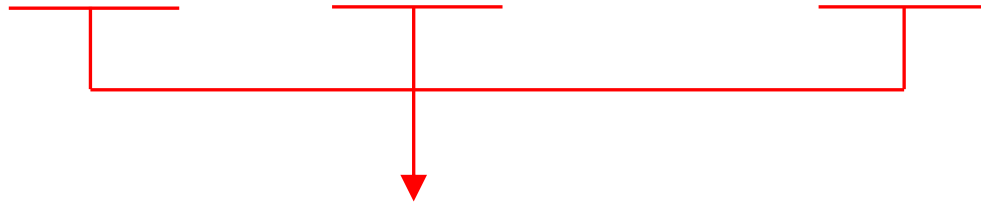
$AR(p)$

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p} + a_t$$

AR 모형과 회귀분석의 차이

회귀분석

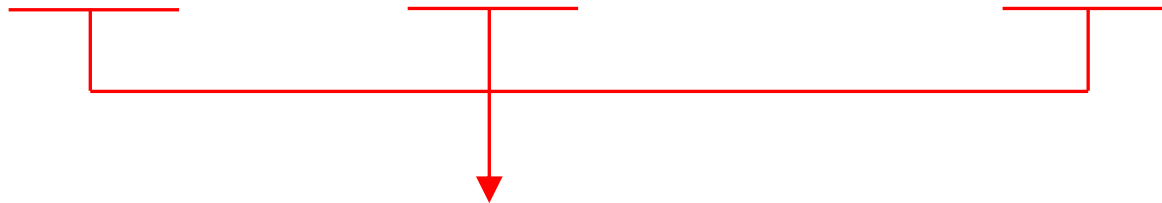
$$Y = \phi_1 x_1 + \phi_2 x_2 + \cdots + \phi_p x_p + a$$



피쳐(자기 자신에 영향을 주는 변수)

$AR(p)$

$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p} + a_t$$



과거의 자기 자신

AR모형을 통해 추정해야할 모수

i) $\mu = 0$ 인 경우
$$Y_t = \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p} + a_t$$

ii) $\mu \neq 0$ 인 경우
$$Y_t = \mu + \phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p} + a_t$$

추정 모수 $p+2$ 개
$$\mu, \phi_1, \phi_2, \cdots, \phi_p, \sigma_a^2$$

이동 평균 과정(Moving Average process, MA)

- 현 시점의 상태를 과거 시점의 오차들,
즉 **과거의 오차로 현재 관측값을 나타냄**

$$MA(1) \quad Y_t = a_t - \theta_1 a_{t-1}$$

현 시점(t)의 값 현 시점(t)의 오차 한 시점 이전(t-1)의 오차

$$MA(2) \quad Y_t = a_t - \theta_1 a_{t-1} - \theta_2 a_{t-2}$$

이동 평균 과정(Moving Average process, MA)

$$MA(q) \quad Y_t = a_t - \theta_1 a_{t-1} - \theta_2 a_{t-2} - \cdots - \theta_q a_{t-q}$$

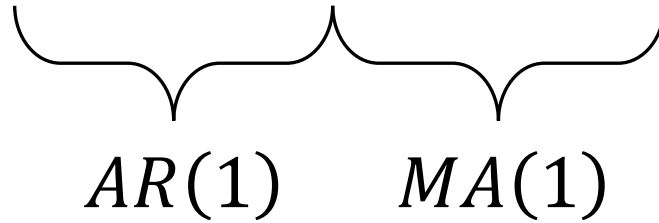
$$Y_t = (1 - \theta_1 \mathbf{B} - \theta_2 \mathbf{B}^2 - \cdots - \theta_q \mathbf{B}^q) a_t$$

$$Y_t = \theta(\mathbf{B}) a_t$$

ARMA(Auto Regressive Moving Average)

$ARMA(1, 1)$

$$Y_t = \phi_1 Y_{t-1} + a_t - \theta_1 a_{t-1}$$



$AR(1) \quad MA(1)$

ARMA(Auto Regressive Moving Average)

$$ARMA(p, q)$$

$$Y_t = \underbrace{\phi_1 Y_{t-1} + \phi_2 Y_{t-2} + \cdots + \phi_p Y_{t-p}}_{AR(p)} + \underbrace{a_t - \theta_1 a_{t-1} - \theta_2 a_{t-2} - \cdots - \theta_q a_{t-q}}_{MA(q)}$$

$$\phi(\mathbf{B})Y_t = \theta(\mathbf{B})a_t$$

ARMA(Auto Regressive Moving Average)

$$ARMA(p, q)$$

$$\phi(\mathbf{B})Y_t = \theta(\mathbf{B})a_t$$

$$Y_t = \phi(\mathbf{B})^{-1}\theta(\mathbf{B})a_t$$

$$= \frac{\theta(\mathbf{B})}{\phi(\mathbf{B})} a_t = \frac{1 - \theta_1 \mathbf{B} - \theta_2 \mathbf{B}^2 - \dots - \theta_q \mathbf{B}^q}{1 - \phi_1 \mathbf{B} - \phi_2 \mathbf{B}^2 - \dots - \phi_p \mathbf{B}^p} a_t$$

ARMAX = 회귀분석 + ARMA

$ARMAX(p, q)$

$$\phi(\mathbf{B})Y_t = \mathbf{X}w + \theta(\mathbf{B})a_t$$

ARMAX = 회귀분석 + ARMA

$$Y_t = \underbrace{\sum_{i=1}^m w_i X_{t-i}}_{\text{회귀분석}} + \underbrace{\sum_{i=1}^p \phi_i Y_{t-i} + \sum_{i=1}^q \theta_i a_{t-i} + a_t}_{ARMA(p, q)}$$

지금까지의 가정 = 약한 정상성(weak stationarity)

$\mu = E(Y_t)$ 평균은 시간 t 에 상관 없이 동일 $\longrightarrow$ 하지만 이 가정이 깨져서 평균이 동일하지
않다면?(nonstationary)

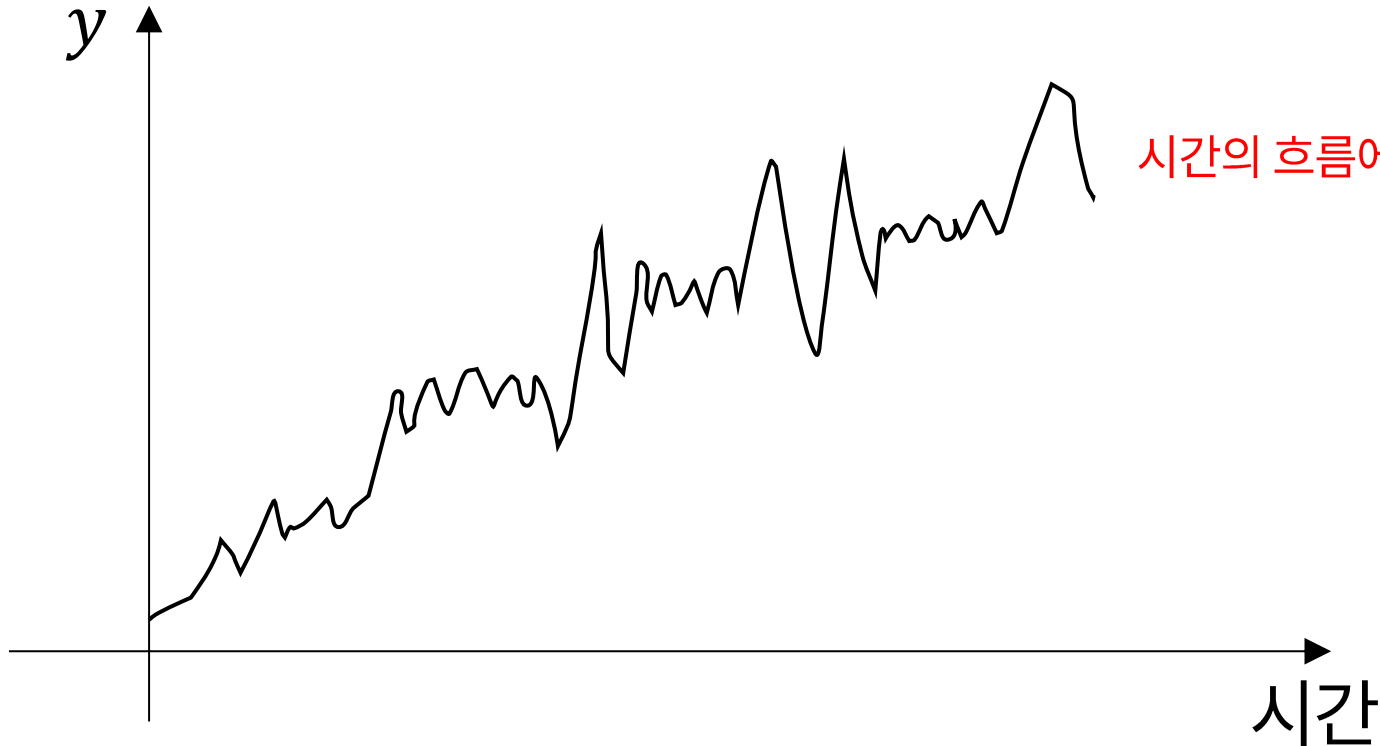
$\sigma^2 = Var(Y_t)$ 분산은 시간 t 에 상관 없이 동일

$\gamma_k = Cov(Y_t, Y_{t+k}) = E[(Y_t - \mu)(Y_{t+k} - \mu)]$
시차 k 에만 의존하고 시점 t 와는 무관

지금까지 사용한 ARMA 모형이 아닌 다른 모형 필요

약한 정상성(weak stationarity) 가정이 깨지는 경우

$\mu = E(Y_t)$ 평균은 시간 t 에 상관 없이 동일 $\longrightarrow$ 하지만 이 가정이 깨져서 평균이 동일하지
않다면?(nonstationary)



시간의 흐름에 따라 시계열 관측값이 추세를 가지고 증가하는 경우

ARIMA(Auto Regressive Integrated Moving Average)

$$ARIMA(p, d, q)$$

$$\phi(\mathbf{B}) \nabla^d Y_t = \theta(\mathbf{B}) a_t \qquad \nabla^d Y_t = (1 - \mathbf{B})^d Y_t$$


ARMA 모형이 정상성을 만족하지 않을 경우 사용

참고: $(1 - \mathbf{B})^{-1} = 1 + \mathbf{B} + \mathbf{B}^2 + \dots$

ARIMA(Auto Regressive Integrated Moving Average)

$$ARIMA(p, d, q)$$

$$\phi(\mathbf{B})\nabla^d Y_t = \theta(\mathbf{B})a_t$$

$$\nabla^d Y_t = \underbrace{\phi_1 \nabla^d Y_{t-1} + \phi_2 \nabla^d Y_{t-2} + \cdots + \phi_p \nabla^d Y_{t-p}}_{AR(p)} + \underbrace{a_t - \theta_1 a_{t-1} - \theta_2 a_{t-2} - \cdots - \theta_q a_{t-q}}_{MA(q)}$$

$\downarrow$
 d

ARIMAX = 회귀분석 + ARIMA

$ARIMAX(p, d, q)$

$$\phi(\mathbf{B})\nabla^d Y_t = \mathbf{X}w + \theta(\mathbf{B})a_t$$

ARIMAX = 회귀분석 + ARIMA

$$ARIMAX(p, d, q)$$

$$\nabla^d Y_t = \underbrace{\sum_{i=1}^m w_i X_{t-i}}_{\text{회귀분석}} + \underbrace{\sum_{i=1}^p \phi_i \nabla^d Y_{t-i} + \sum_{j=1}^q \theta_j a_{t-j} + a_t}_{ARIMA(p, d, q)}$$

데이터에 계절성(seasonal)이 나타나면?

계절성이란 데이터가 특정 주기 s 에 따라 반복되는 경향이 나타나는 경향이 있는 경우

ex) 매년 8월마다 증가하는 아이스크림 판매량(계절성 주기 12개월)

ex) 매년 3월마다 증가하는 노트북 판매량(계절성 주기 12개월)

SARIMA(Seasonal ARIMA)

계절성을 표현하기 위해 등장하는 새로운 연산자 ∇_s

$$\nabla_s Y_t = (1 - B^s)Y_t = Y_t - Y_{t-s}$$

$$\nabla = (1 - B)$$

$$B^s Y_t = Y_{t-s}$$

SARIMA(Seasonal ARIMA)

$SARIMA(p, d, q)(P, D, Q)_s$ ARMA 모형이 계절성을 따른 경우 사용

$$\phi(\mathbf{B})\Phi(\mathbf{B}^s)\nabla^d\nabla_s^DY_t = \theta(\mathbf{B})\Theta(\mathbf{B}^s)a_t$$

$$\phi(\mathbf{B})\Phi(\mathbf{B}^s)(1 - \mathbf{B})^d(1 - \mathbf{B}^s)^DY_t = \theta(\mathbf{B})\Theta(\mathbf{B}^s)a_t$$

SARIMA(Seasonal ARIMA)

$$SARIMA(0, 1, 1)(0, 1, 1)_{12}$$

ARMA 모형이 계절성을 따른 경우 사용

0이므로 이 부분 사라짐

$$\phi(B)\Phi(B^1)\nabla^1\nabla_{12}^1 Y_t = \theta(B)\Theta(B^{12})a_t$$

$$\phi(B)\Phi(B^1)(1-B)^1(1-B^{12})^1 Y_t = \theta(B)\Theta(B^{12})a_t$$

SARIMA(Seasonal ARIMA)

$SARIMA(0, 1, 1)(0, 1, 1)_{12}$ ARMA 모형이 계절성을 따른 경우 사용

$$\nabla^1 \nabla_{12}^1 Y_t = \theta(\mathbf{B}) \Theta(\mathbf{B}^{12}) a_t$$

$$(1 - \mathbf{B})^1 (1 - \mathbf{B}^{12})^1 Y_t = \theta(\mathbf{B}) \Theta(\mathbf{B}^{12}) a_t$$

SARIMA(Seasonal ARIMA)

$$SARIMA(0, 1, 1)(0, 1, 1)_{12} \quad \nabla \nabla_{12}^1 Y_t = \theta(B) \Theta(B^{12}) a_t$$

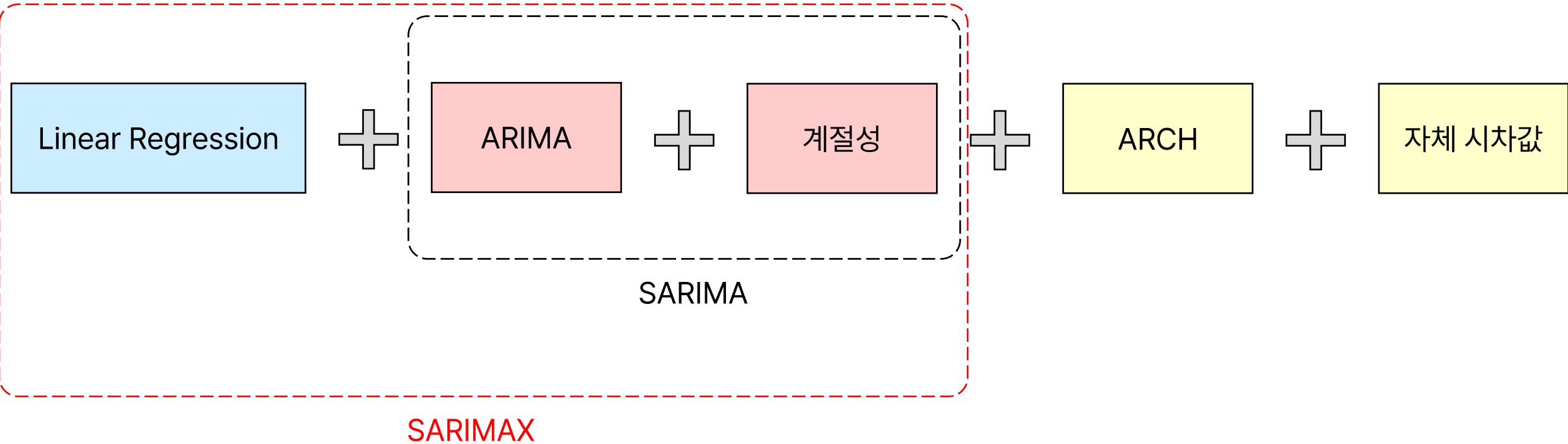
$$\longleftrightarrow (1 - B)(1 - B^{12})Y_t = (1 - \theta B)(1 - \Theta B^{12})a_t$$

$$\longleftrightarrow (1 - B^{12} - B + B^{13})Y_t = (1 - \Theta B^{12} - \theta B + \theta \Theta B^{13})a_t$$

$$\longleftrightarrow (1 - B^{12} - B + B^{13})Y_t = (1 - \Theta B^{12} - \theta B + \theta \Theta B^{13})a_t$$

$$\longleftrightarrow Y_t - Y_{t-12} - Y_{t-1} + Y_{t-13} = a_t - \Theta a_{t-12} - \theta a_{t-1} + \theta \Theta a_{t-13}$$

SARIMAX(Seasonal ARIMAX)



SARIMAX = 회귀분석 + SARIMA

$$\phi(\mathbf{B})\Phi(\mathbf{B}^s)\nabla^d\nabla_s^D Y_t = \mathbf{X}w + \theta(\mathbf{B})\Theta(\mathbf{B}^s)a_t$$

SARIMAX = 회귀분석 + SARIMA

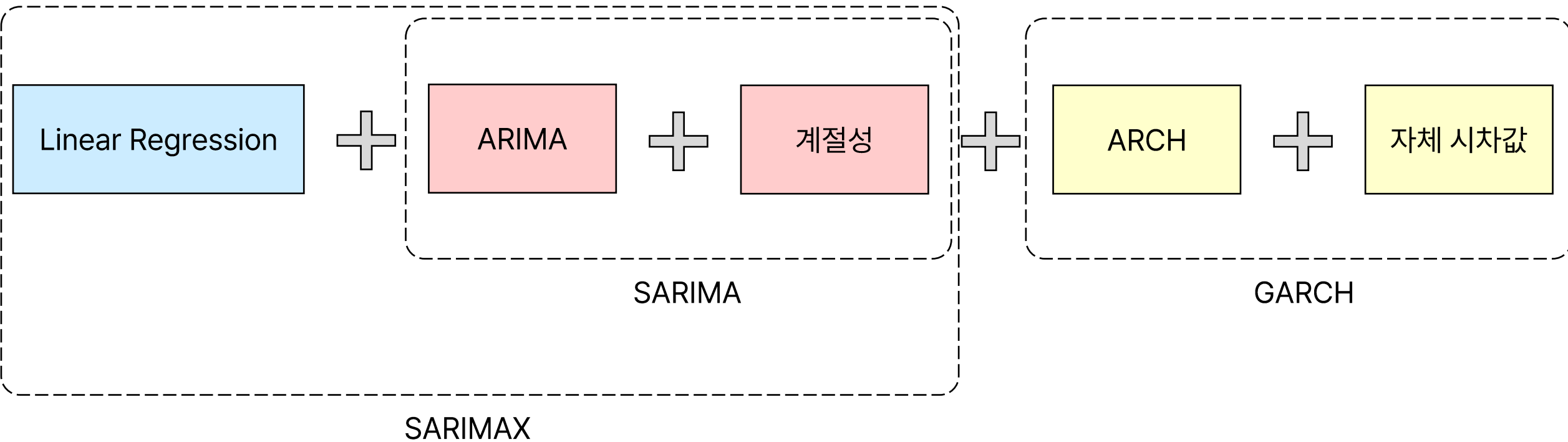
$$\nabla^d \nabla_s^D Y_t = \underbrace{\sum_{i=1}^m w_i X_{t-i}}_{\text{회귀분석}} + \underbrace{\sum_{i=1}^p \phi_i \nabla^d \nabla_s^D Y_{t-i} + \sum_{i=1}^q \theta_i a_{t-i} + \sum_{i=1}^P \Phi_i \nabla^d \nabla_s^D Y_{t-i} + \sum_{i=1}^Q \Theta_i a_{t-i} + a_t}_{SARIMA(p, d, q)(P, D, Q)_s}$$

AI/BigData 인텐시브 과정

Section 2. ARIMA

Section 2-3. ARIMA 실습

시계열 분석



pandas 데이터프레임 주기 변환

In [27]: ▶

1 y_tn

Out[27]: DateTimeIndex

2005-02-16 17:00:00	1010
2005-02-16 18:00:00	1110
2005-02-16 19:00:00	1300
2005-02-16 20:00:00	1542
2005-02-16 21:00:00	1294
...	
2005-03-26 00:00:00	1534
2005-03-26 01:00:00	1488
2005-03-26 02:00:00	1361
2005-03-26 03:00:00	1360
2005-03-26 04:00:00	1336

Name: PT08_S4, Length: 900, dtype: int64

현재 주기가 설정되어 있지 않은 것 확인

데이터프레임 주기 변환

In [28]:



```
1 X_train = X_tn.asfreq('H')
2 y_train = y_tn.asfreq('H')
```

'D' = Day
'W' = Week
'H' = Hour

In [29]:



```
1 X_test = X_te.asfreq('H')
2 y_test = y_te.asfreq('H')
```

In [30]:



```
1 y_test
```

Out[30]:

```
DateTimeIndex
2005-03-26 05:00:00    1309
2005-03-26 06:00:00    1293
2005-03-26 07:00:00    1325
2005-03-26 08:00:00    1458
2005-03-26 09:00:00    1522
...
2005-03-30 04:00:00    1073
2005-03-30 05:00:00    1109
2005-03-30 06:00:00    1164
2005-03-30 07:00:00    1457
2005-03-30 08:00:00    1533
```

주기 설정된 것 확인

```
Freq: H, Name: PT08_S4, Length: 100, dtype: int64
```

SARIMAX 모형 학습

$$SARIMAX(p, d, q)(P, D, Q)_s$$

```
In [31]: ▶ 1 sarimax = ARIMA(endog=타겟y_train, exog=피쳐X_train, order=(1,1,0), seasonal_order=(0,1,1,24))  
2 sarimax_fit = sarimax.fit()
```

$$SARIMAX(1, 1, 0)(0, 1, 1)_{24}$$

SARIMAX 모형 학습

$$SARIMAX(p, d, q)(P, D, Q)_s \triangleright y = w_1 x_1 + w_2 x_2 + SARIMA(1, 1, 0)(0, 1, 1)_{24}$$

SARIMAX 모형

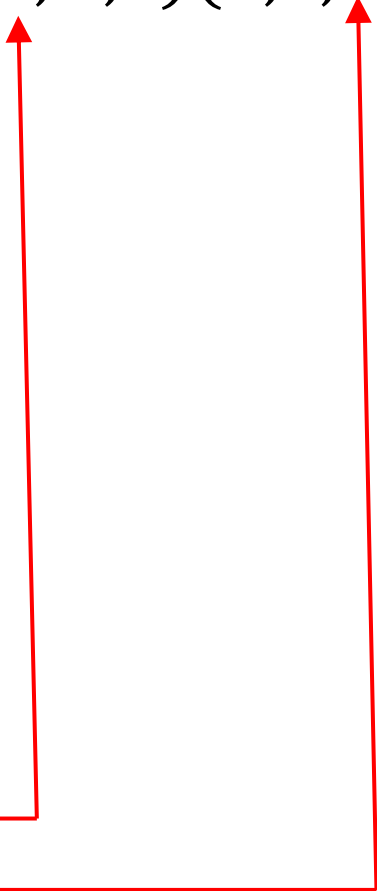
```
1 sarimax_fit.summary()
```

$$y = -16.14x_1 + 24.58x_2 + SARIMA(1, 1, 0)(0, 1, 1)_{24}$$

SARIMAX Results

Dep. Variable:	PT08_S4	No. Observations:	900
Model:	ARIMA(1, 1, 0)x(0, 1, [1], 24)	Log Likelihood	-5244.604
Date:	Wed, 31 Jan 2024	AIC	10499.208
Time:	23:07:14	BIC	10523.079
Sample:	02-16-2005	HQIC	10508.340
	- 03-26-2005		
Covariance Type:	opg		

	coef	std err	z	P> z	[0.025	0.975]
T	-16.1453	3.231	-4.996	0.000	-22.479	-9.812
AH	24.5826	3.539	6.946	0.000	17.646	31.520
ar.L1	-0.0230	0.026	-0.885	0.376	-0.074	0.028
ma.S.L24	-0.8801	0.015	-59.336	0.000	-0.909	-0.851
sigma2	9040.4793	302.297	29.906	0.000	8447.988	9632.971



train 데이터로 예측

In [33]:



```
1 sarimax_fit.predict()
```

Out[33]:

DateTimeIndex

2005-02-16 17:00:00	-121.174294
2005-02-16 18:00:00	1022.755962
2005-02-16 19:00:00	1115.900212
2005-02-16 20:00:00	1306.084612
2005-02-16 21:00:00	1552.961913

...

2005-03-26 00:00:00	1427.921306
2005-03-26 01:00:00	1489.220439
2005-03-26 02:00:00	1425.691601
2005-03-26 03:00:00	1336.854742
2005-03-26 04:00:00	1337.022572

Freq: H, Name: predicted_mean, Length: 900, dtype: float64

test 데이터로 예측

```
1 sarimax_fit.forecast(steps=len(X_test), exog=X_test)
```

```
2005-03-26 05:00:00    1317.998473
2005-03-26 06:00:00    1334.624957
2005-03-26 07:00:00    1413.269919
2005-03-26 08:00:00    1767.346875
2005-03-26 09:00:00    1956.451180
...
2005-03-30 04:00:00    1571.202451
2005-03-30 05:00:00    1576.185181
2005-03-30 06:00:00    1576.608120
2005-03-30 07:00:00    1660.099109
2005-03-30 08:00:00    1998.104528
Freq: H, Name: predicted_mean, Length: 100, dtype: float64
```

```
1 y_test
```

```
: DateTimeIndex
2005-03-26 05:00:00    1309
2005-03-26 06:00:00    1293
2005-03-26 07:00:00    1325
2005-03-26 08:00:00    1458
2005-03-26 09:00:00    1522
...
2005-03-30 04:00:00    1073
2005-03-30 05:00:00    1109
2005-03-30 06:00:00    1164
2005-03-30 07:00:00    1457
2005-03-30 08:00:00    1533
Freq: H, Name: PT08_S4, Length: 100, dtype: int64
```

AI/BigData 인텐시브 과정

Section 3. ARCH, GARCH

Section 3-1. ARCH, GARCH의 개념

ARIMA 가정이 깨지면?

$$\sigma_t^2 = Var(a_t | \text{과거 데이터})$$

등분산성 가정이 깨지면?



분산(변동성) 추정에 arch, garch 모형 사용

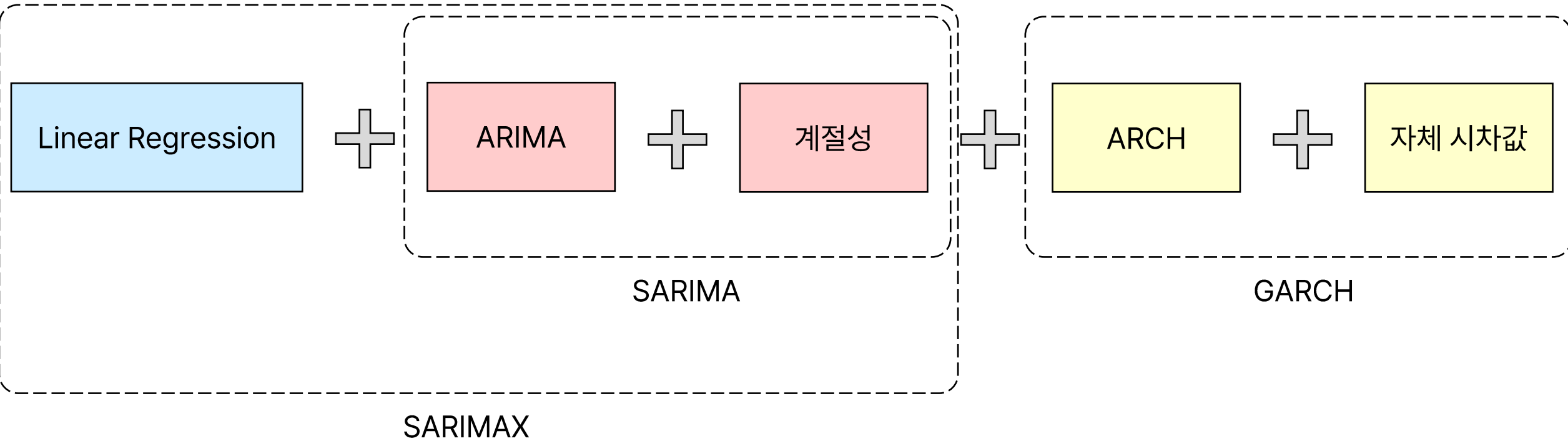
등분산성 가정이 깨진다는 말의 의미

등분산성 가정이 깨지면? —————> 분산이 시간에 따라 변함

ex) 주식, 재무 시계열 데이터는 분산이 과거 데이터에 의존한다는 특징이 있음

—————> ARCH 모형 제안

시계열 분석



ARCH(autoregressive conditional heteroskedasticity)

$ARCH(s)$ $a_t = \sigma_t e_t$ e_t 는 iid를 만족하며, 평균 0, 분산 1인 수열

$$\sigma_t^2 = \alpha_0 + \alpha_1 a_{t-1}^2 + \alpha_2 a_{t-2}^2 + \cdots + \alpha_s a_{t-s}^2$$

$$\alpha_0 > 0,$$

$$\alpha_i \geq 0 \text{ for } i = 1, \dots, s-1,$$

$$\alpha_s > 0$$

$$\sum_{i=1}^s \alpha_i < 1$$

GARCH(generalized ARCH)

$GARCH(1, 1)$ $a_t = \sigma_t e_t$ e_t 는 iid를 만족하며, 평균 0, 분산 1인 수열

$$\sigma_t^2 = \alpha_0 + \alpha_1 a_{t-1}^2 + \beta_1 \sigma_{t-1}^2$$

GARCH(generalized ARCH)

$GARCH(s, r)$ $a_t = \sigma_t e_t$ e_t 는 iid를 만족하며, 평균 0, 분산 1인 수열

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^s \alpha_i a_{t-i}^2 + \sum_{j=1}^r \beta_j \sigma_{t-j}^2$$

ARIMA + GARCH

$$\nabla^d Y_t = \underbrace{\sum_{i=1}^p \phi_i \nabla^d Y_{t-i}}_{ARIMA(p, d, q)} + \underbrace{\sum_{j=1}^q \theta_j a_{t-j} + a_t}_{GARCH(s, r)}$$

$$a_t = \sigma_t e_t$$

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^s \alpha_i a_{t-i}^2 + \sum_{j=1}^r \beta_j \sigma_{t-j}^2$$

SARIMAX-GARCH = 회귀분석 + SARIMA + GARCH

$$\nabla^d \nabla_s^D Y_t = \underbrace{\sum_{i=1}^m w_i X_{t-i}}_{\text{회귀분석}} + \underbrace{\sum_{i=1}^p \phi_i \nabla^d \nabla_s^D Y_{t-i} + \sum_{i=1}^q \theta_i a_{t-i} + \sum_{i=1}^P \Phi_i \nabla^d \nabla_s^D Y_{t-i} + \sum_{i=1}^Q \Theta_i a_{t-i}}_{\text{SARIMA}(p, d, q)(P, D, Q)_s} + \underbrace{a_t}_{\text{GARCH}(s, r)}$$

$$a_t = \sigma_t e_t \quad \sigma_t^2 = \alpha_0 + \sum_{i=1}^s \alpha_i a_{t-i}^2 + \sum_{j=1}^r \beta_j \sigma_{t-j}^2$$

AI/BigData 인텐시브 과정

Section 3. ARCH, GARCH

Section 3-2. ARCH, GARCH 실습

ARCH 학습

In [36]: ▶

```
1 arch = arch_model(y_train, vol='ARCH', p=1)
2 arch_fit = arch.fit()
```

Iteration:	1,	Func. Count:	5,	Neg. LLF:	6145.472529512024
Iteration:	2,	Func. Count:	10,	Neg. LLF:	6141.004542180426
Iteration:	3,	Func. Count:	14,	Neg. LLF:	6129.2687634324775
Iteration:	4,	Func. Count:	18,	Neg. LLF:	6120.179168927249
Iteration:	5,	Func. Count:	22,	Neg. LLF:	6119.610661065649
Iteration:	6,	Func. Count:	26,	Neg. LLF:	6119.588002745563
Iteration:	7,	Func. Count:	30,	Neg. LLF:	6119.587793126943
Iteration:	8,	Func. Count:	34,	Neg. LLF:	6119.587789379819
Iteration:	9,	Func. Count:	38,	Neg. LLF:	6119.587783770034
Iteration:	10,	Func. Count:	42,	Neg. LLF:	6119.587767089447

ARCH 예측

```
In [37]: ▶ 1 arch_fit.forecast
```

Out[37]: <bound method ARCHModelFixedResult.forecast of Constant Mean - ARCH Model Results

```
=====
Dep. Variable:          PT08_S4    R-squared:                0.000
Mean Model:            Constant Mean  Adj. R-squared:          0.000
Vol Model:              ARCH          Log-Likelihood:        -6114.03
Distribution:           Normal        AIC:                    12234.1
Method:                Maximum Likelihood  BIC:                    12248.5
                                     No. Observations:          900
Date:                  Thu, Feb 01 2024  Df Residuals:          899
Time:                  00:11:12         Df Model:              1
                                     Mean Model
=====
              coef    std err          t      P>|t|      95.0% Conf. Int.
-----
mu          1051.2059    25.554     41.136    0.000 [1.001e+03,1.101e+03]
              Volatility Model
=====
              coef    std err          t      P>|t|      95.0% Conf. Int.
-----
omega       1.0041e+04   3794.373     2.646  8.138e-03 [2.604e+03,1.748e+04]
alpha[1]    1.0000        0.145     6.897  5.307e-12 [ 0.716,  1.284]
=====
```

ARCH 예측

```
In [38]: 1 pred_var = arch_fit.forecast(horizon=len(y_test))
```

```
In [39]: 1 pred_var.variance.values
```

```
Out[39]: array([[ 91148.64734654, 101189.59126678, 111230.53518701,
                  121271.47910724, 131312.42302748, 141353.36694771,
                  151394.31086794, 161435.25478817, 171476.19870841,
                  181517.14262864, 191558.08654887, 201599.03046911,
                  211639.97438934, 221680.91830957, 231721.86222981,
                  241762.80615004, 251803.75007027, 261844.6939905 ,
```

$$ARCH(s)$$

$$a_t = \sigma_t e_t$$

ARCH 예측

In [40]: ▶ 1 $\text{pred_arch} = \text{np.sqrt}(\text{pred_var.variance.values}[0])$

In [41]: ▶ 1 pred_arch $\sigma_t^2 = \alpha_0 + \alpha_1 a_{t-1}^2 + \alpha_2 a_{t-2}^2 + \cdots + \alpha_s a_{t-s}^2$

Out[41]: array([301.90834262, 318.1031142 , 333.51242134, 348.24054776,
362.37056038, 375.96990165, 389.0942185 , 401.79006308,
414.09684702, 426.04828673, 437.67349308, 448.99780675,
460.04344837, 470.83003123, 481.37497051, 491.69381341,
501.80050824, 511.7076255 , 521.42654124, 530.96759019,

$$a_t = \sigma_t e_t \quad \boxed{\sigma_t^2} = \alpha_0 + \alpha_1 a_{t-1}^2 + \alpha_2 a_{t-2}^2 + \cdots + \alpha_s a_{t-s}^2$$

루트 취해야함

GARCH 학습

```
In [42]: ▶ 1 garch = arch_model(y_train, vol='GARCH', p=15, q=15)
          2 garch_fit = garch.fit()
```

C:\Users\stocic\conda\envs\Wpy3_10_9\lib\site-packages\arch\univariate\base.py:31
poorly scaled, which may affect convergence of the optimizer when
estimating the model parameters. The scale of y is 7.674e+04. Parameter
estimation work better when this value is between 1 and 1000. The recommended
rescaling is $0.1 * y$.

This warning can be disabled by either rescaling y before initializing the
model or by setting `rescale=False`.

```
warnings.warn(
```

Iteration:	1,	Func. Count:	34,	Neg. LLF:	6179.791297746277
Iteration:	2,	Func. Count:	67,	Neg. LLF:	6123.397145478621
Iteration:	3,	Func. Count:	101,	Neg. LLF:	6748.969787008376
Iteration:	4,	Func. Count:	137,	Neg. LLF:	7869.9729490697755
Iteration:	5,	Func. Count:	172,	Neg. LLF:	6110.230369907059
Iteration:	6,	Func. Count:	206,	Neg. LLF:	6103.991024259718

사실 arch와 garch는 평균이 0이고
변동성이 큰 데이터에 많이 적용하지
만 이번 실습은 그냥 진행

GARCH 예측

```
In [43]: 1 pred_var = garch_fit.forecast(horizon=len(y_test))
```

```
In [44]: 1 pred_garch = np.sqrt(pred_var.variance.values[0])
```

```
In [45]: 1 pred_garch
```

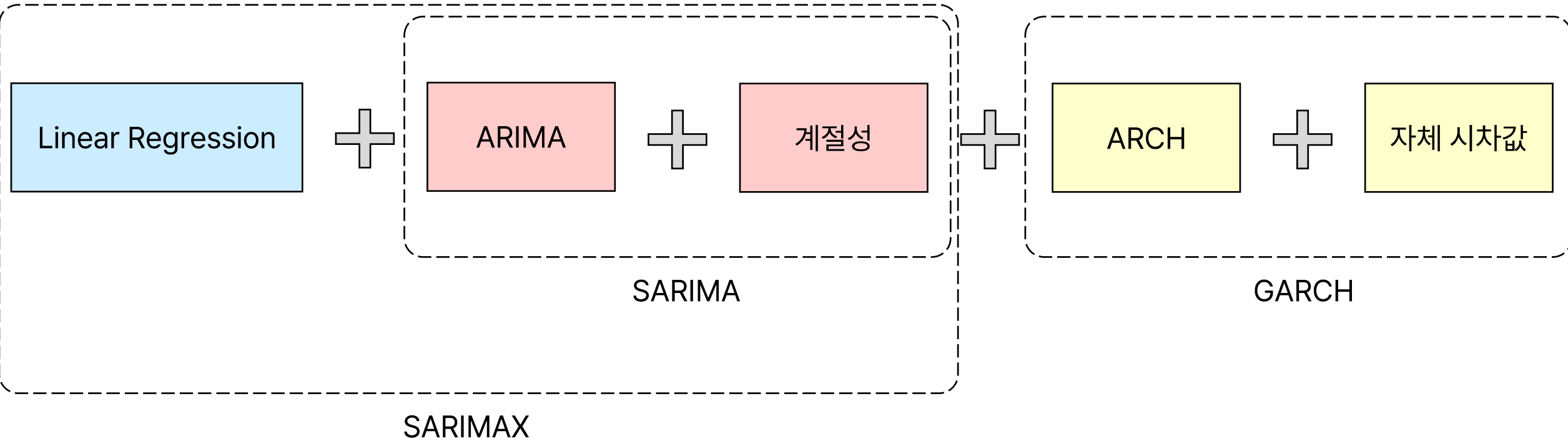
```
Out[45]: array([323.34392779, 366.90926527, 430.52526383, 495.65456036,  
                534.65102528, 548.28102366, 553.70127821, 550.0473425 ,  
                551.1281603 , 549.32863922, 541.3882488 , 533.90673733,  
                525.93265168, 520.85054571, 518.30196955, 519.63405863,  
                525.21633271, 533.17562529, 541.45811618, 549.41416524,  
                556.39430615, 562.85063339, 568.62273887, 573.33716529,  
                577.13559846, 580.07549 , 582.43577726, 584.43854527,
```

AI/BigData 인텐시브 과정

Section 3. ARCH, GARCH

Section 3-3. SARIMAX + GARCH 실습

시계열 분석



SARIMAX 학습

```
In [63]: ▶ 1 sarimax = ARIMA(endog=y_train, exog=X_train, order=(1,1,0), seasonal_order=(0,1,1,12))  
        2 sarimax_fit = sarimax.fit()
```

$$SARIMAX(p, d, q)(P, D, Q)_s$$

$$\triangleright y = w_1 x_T + w_2 x_{AH} + SARIMA(1, 1, 0)(0, 1, 1)_{12} + \boxed{a_t}$$

?

SARIMAX 학습

```

1 # sarima 모형 예러
2 sarimax_residual = sarimax_fit.resid

```

```

1 sarimax_residual

```

```

5]: DateTimeIndex
2005-02-16 17:00:00    1015.103972
2005-02-16 18:00:00     97.774043
2005-02-16 19:00:00    189.351137
2005-02-16 20:00:00    241.135164
2005-02-16 21:00:00   -249.359674
...
2005-03-26 00:00:00    111.494728
2005-03-26 01:00:00    -46.802530
2005-03-26 02:00:00   -101.757225
2005-03-26 03:00:00     37.279488
2005-03-26 04:00:00    -22.630143
Freq: H, Length: 900, dtype: float64

```

$$SARIMAX(p, d, q)(P, D, Q)_s$$



$$y = w_1 x_T + w_2 x_{AH} + SARIMA(1, 1, 0)(0, 1, 1)_{12} + a_t$$

SARIMAX 모형의 잔차

SARIMAX + GARCH

1	sarimax_residual
: DateTimeIndex	
2005-02-16 17:00:00	1015.103972
2005-02-16 18:00:00	97.774043
2005-02-16 19:00:00	189.351137
2005-02-16 20:00:00	241.135164
2005-02-16 21:00:00	-249.359674
...	
2005-03-26 00:00:00	111.494728
2005-03-26 01:00:00	-46.802530
2005-03-26 02:00:00	-101.757225
2005-03-26 03:00:00	37.279488
2005-03-26 04:00:00	-22.630143
Freq: H, Length: 900, dtype: float64	

$$y = w_1 x_T + w_2 x_{AH} + SARIMA(1, 1, 0)(0, 1, 1)_{12} + a_t$$

SARIMAX 모형의 잔차를 활용해 GARCH 모형 학습

잔차 변환

```
1 sarimax_residual
```

```
: DateTimeIndex
2005-02-16 17:00:00    1015.103972
2005-02-16 18:00:00     97.774043
2005-02-16 19:00:00    189.351137
2005-02-16 20:00:00    241.135164
2005-02-16 21:00:00   -249.359674
...
2005-03-26 00:00:00    111.494728
2005-03-26 01:00:00    -46.802530
2005-03-26 02:00:00   -101.757225
2005-03-26 03:00:00     37.279488
2005-03-26 04:00:00    -22.630143
Freq: H, Length: 900, dtype: float64
```



```
1 adjusted_constant = 0.1
```

```
2 adj_sarimax_residual = sarimax_residual*adjusted_constant
```

```
1 adj_sarimax_residual.describe()
```

```
: count    900.000000
   mean      0.105443
   std      12.104562
   min     -53.930941
   25%     -5.470846
   50%     -0.338875
   75%      4.947958
   max     101.510397
   dtype: float64
```

GARCH에 사용하는 데이터는 1부터 1000사이의 값이 좋은데, 해당 데이터는 10000이상의 값과 음수값이 포함되어 있으므로 오른쪽과 같이 기존 값에 0.1을 곱한 값으로 변환하며 이는 추후에 원래 값으로 변환 필요

SARIMAX + GARCH(GARCH 학습)

In [69]: ▶

```
1 garch = arch_model(adj_sarimax_residual, vol='GARCH', p=20, q=20)
2 garch_fit = garch.fit()
```

Iteration:	1,	Func. Count:	44,	Neg. LLF:	3474.826064472155
Iteration:	2,	Func. Count:	88,	Neg. LLF:	3447.6589425535726
Iteration:	3,	Func. Count:	132,	Neg. LLF:	3564.2744709286644
Iteration:	4,	Func. Count:	177,	Neg. LLF:	3674.7326830177617
Iteration:	5,	Func. Count:	222,	Neg. LLF:	3523.8581548232287

↓
SARIMAX의 잔차를 활용해 GARCH 모형 학습

$$y = w_1 x_T + w_2 x_{AH} + SARIMA(1, 1, 0)(0, 1, 1)_{12} + a_t$$

$$\triangleright y = w_1 x_T + w_2 x_{AH} + SARIMA(1, 1, 0)(0, 1, 1)_{12} + GARCH(p, q)$$

SARIMAX로 mu 예측

```
1 pred_mu = sarimax_fit.forecast(steps=len(X_test), exog=X_test)
2 pred_mu
```

```
2005-03-26 05:00:00    1343.640235
2005-03-26 06:00:00    1424.351499
2005-03-26 07:00:00    1527.959909
2005-03-26 08:00:00    1668.701510
2005-03-26 09:00:00    1635.782565
```

...

```
2005-03-30 04:00:00    1373.488620
2005-03-30 05:00:00    1384.862011
2005-03-30 06:00:00    1464.676293
2005-03-30 07:00:00    1568.636243
2005-03-30 08:00:00    1708.312443
```

Freq: H, Name: predicted_mean, Length: 100, dtype: float64

$$y = w_1 x_T + w_2 x_{AH} + \boxed{SARIMA(1, 1, 0)(0, 1, 1)_{12}} + GARCH(20, 20)$$

이 부분(SARIMAX)을 활용해 mu 예측

SARIMAX + GARCH(GARCH로 residual 예측)

```
In [71]: ▶ 1 adj_forecast_residual = garch_fit.forecast(horizon=len(y_test))  
        2 adj_forecast_residual
```

```
Out[71]: <arch.univariate.base.ARCHModelForecast at 0x194cb49f430>
```

```
In [72]: ▶ 1 adj_forecast_residual.variance.values
```

```
Out[72]: array([[ 56.00184643, 161.44358636, 180.5695313 , 327.33268652,  
                299.98626623, 253.81899433, 173.72654643, 156.1289081 ,  
                107.766304 , 93.20824647, 70.42514577, 71.73911861,  
                84.99722641, 145.92588982, 182.96520387, 256.18251155,
```

$$Y = 0.1X$$

원래 값으로 복원

$$\sigma_Y^2 = \text{Var}(0.1X) = 0.1^2 \text{Var}(X) = 0.1^2 \sigma_X^2$$

$$\sigma_Y = \sqrt{0.1^2 \sigma_X^2} = 0.1 \sigma_X \longrightarrow \sigma_X = \sigma_Y / 0.1$$

In [113]: ▶ 1 forecast_residual = np.sqrt(adj_forecast_residual.variance.values)/adjusted_constant

In [114]: ▶ 1 forecast_residual

Out[114]: array([[74.83438142, 127.06045268, 134.37616281, 180.92337785,
173.20111611, 159.31697785, 131.80536652, 124.95155385,
103.81055052, 96.544418 , 83.91969123, 84.69894841,

$$a_t = \sigma_t e_t \quad \boxed{\sigma_t^2} = \alpha_0 + \alpha_1 a_{t-1}^2 + \alpha_2 a_{t-2}^2 + \cdots + \alpha_s a_{t-s}^2$$



루트 취해야함

pandas series 형태로 변환

```
In [115]: ▶ 1 pred_residual = pd.Series(forecast_residual[-1, :], index=y_test.index)
```

```
In [116]: ▶ 1 pred_residual
```

```
Out[116]: DateTimeIndex
           2005-03-26 05:00:00    74.834381
           2005-03-26 06:00:00   127.060453
           2005-03-26 07:00:00   134.376163
           2005-03-26 08:00:00   180.923378
           2005-03-26 09:00:00   173.201116
                        ...
           2005-03-30 04:00:00   119.250714
           2005-03-30 05:00:00   119.422475
           2005-03-30 06:00:00   120.681466
           2005-03-30 07:00:00   122.615618
           2005-03-30 08:00:00   124.651331
Freq: H, Length: 100, dtype: float64
```

SARIMAX + GARCH 예측 결과

```
In [118]: 1 e_t = np.random.randn(len(X_test))
          2 y_hat = pred_mu + pred_residual*e_t
```

```
In [119]: 1 y_hat
```

```
Out[119]: 2005-03-26 05:00:00    1211.267433
          2005-03-26 06:00:00    1357.196775
          2005-03-26 07:00:00    1527.157100
          2005-03-26 08:00:00    1661.409901
          2005-03-26 09:00:00    1518.782096
          ...
          2005-03-30 04:00:00    1486.339632
          2005-03-30 05:00:00    1314.699823
          2005-03-30 06:00:00    1542.921348
          2005-03-30 07:00:00    1429.263226
          2005-03-30 08:00:00    1631.656815
          Freq: H, Length: 100, dtype: float64
```

$$a_t = \sigma_t e_t$$

```
In [120]: 1 y_test
```

```
Out[120]: DateTimeIndex
          2005-03-26 05:00:00    1309
          2005-03-26 06:00:00    1293
          2005-03-26 07:00:00    1325
          2005-03-26 08:00:00    1458
          2005-03-26 09:00:00    1522
          ...
          2005-03-30 04:00:00    1073
          2005-03-30 05:00:00    1109
          2005-03-30 06:00:00    1164
          2005-03-30 07:00:00    1457
          2005-03-30 08:00:00    1533
          Freq: H, Name: PT08_S4, Length: 100, dtype: int64
```

감사합니다.

Q & A